

Estructuras de Datos en Inteligencia Artificial

Un recorrido integral: de las estructuras básicas de Python a grafos y algoritmos de búsqueda

Cátedra:	Programación aplicada a Inteligencia Artificial
Modalidad:	Informe técnico-académico
Nivel:	Universitario — introductorio a intermedio
Autor:	Prof. Cátedra IA
Contenido:	list · tuple · dict · set · Counter · deque · pila · cola · heap árbol binario · árbol n-ario · grafo · Dataset · DataFrame · BFS · DFS

“En Inteligencia Artificial, elegir la estructura de datos correcta suele ser más determinante que elegir el algoritmo: la primera define lo que el segundo puede hacer, en qué tiempo y con qué memoria.”

1. Introducción

Este informe amplía el estudio de las estructuras de datos que un estudiante o profesional de IA debe dominar. No se trata sólo de conocer sus sintaxis, sino de entender **cuándo cada una es la elección correcta**. Un modelo de aprendizaje profundo, un agente de búsqueda o un sistema de recomendación no se distinguen únicamente por sus algoritmos, sino por las estructuras que sostienen esos algoritmos por debajo.

Cubriremos tres grandes familias:

- **Colecciones básicas de Python:** list, tuple, dict, set, Counter, deque.
- **Estructuras lineales especializadas:** pilas, colas, heaps.
- **Estructuras no lineales:** árboles (binarios y n-arios), grafos, y sus recorridos fundamentales (BFS, DFS).
- **Estructuras del ecosistema de datos:** Dataset y DataFrame.

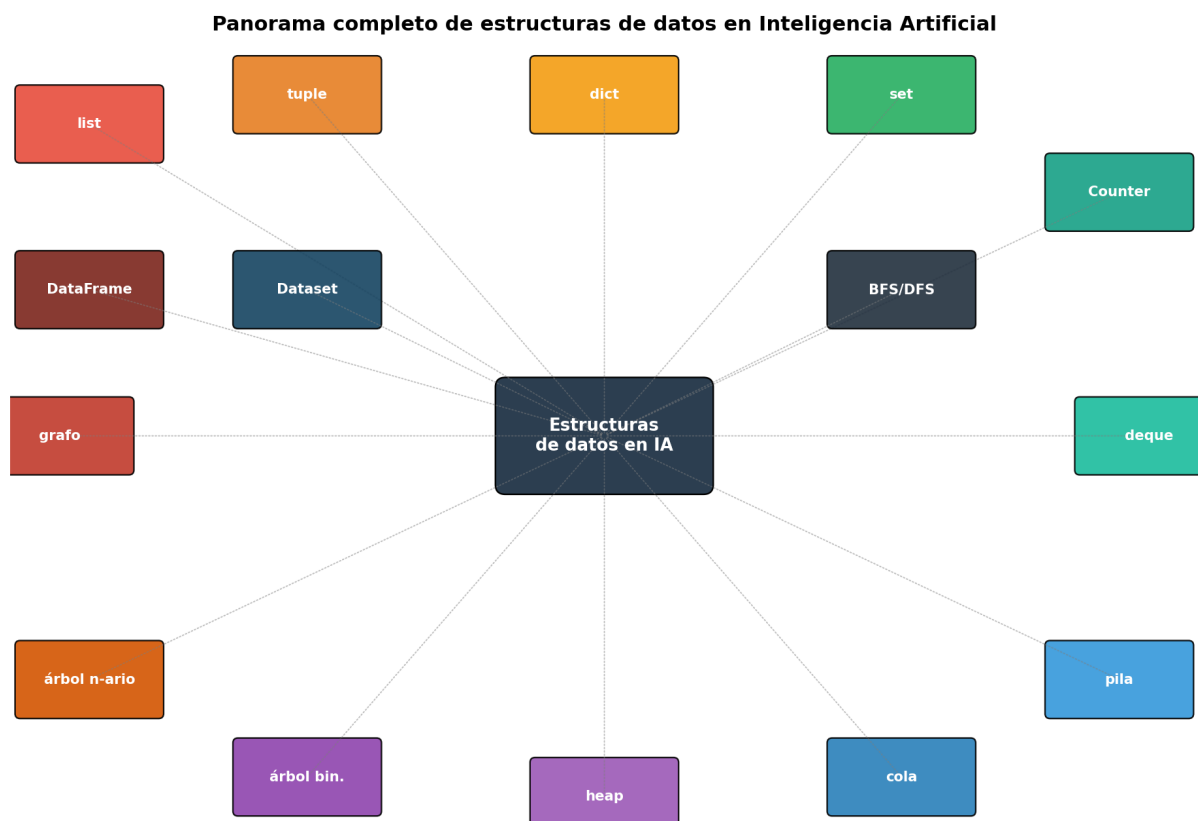


Figura 1. Panorama general de estructuras estudiadas.

2. list — secuencia mutable

Una **list** es una secuencia **ordenada**, **mutable** y heterogénea. Se implementa como arreglo dinámico de punteros, con acceso por índice $O(1)$ y búsqueda por valor $O(n)$.

Uso en IA

```
predicciones = [0.87, 0.92, 0.11, 0.65]
predicciones.append(0.78)
loss_history = [] # curva de pérdida por época
```

Se usa para secuencias de tokens, historiales de pérdida, batches temporales antes de convertir a tensores, y como contenedor genérico en scripts de preprocesamiento.

3. tuple — secuencia inmutable

Una **tuple** es ordenada, **inmutable** y hashable (si sus elementos lo son). Puede usarse como clave de dict o elemento de set.

```
muestra = (features, label) # par supervisado
shape = (224, 224, 3) # forma de imagen
return loss, accuracy # retorno múltiple (tupla implícita)
```

Presente en pares (x, y), formas de tensores, coordenadas, y retornos múltiples de funciones. Los DataLoaders de PyTorch entregan tuplas.

4. dict — mapeo clave→valor

Tabla hash: acceso, inserción y borrado promedio $O(1)$. Preserva orden de inserción desde Python 3.7. Las claves deben ser hashables.

```
config = {"lr": 1e-3, "batch_size": 32}
vocab = {"hola": 0, "mundo": 1}
torch.save(model.state_dict(), "m.pt")
```

Es la estructura más versátil del ecosistema Python: configs, vocabularios, métricas, JSON de API, y los pesos de una red neuronal viajan como dict.

5. set — conjunto sin duplicados

Colección no ordenada de elementos únicos y hashables. Pertenencia $O(1)$. Soporta operaciones de conjunto ($\cup$, $\cap$, $-$).

```
vocab_train = {"perro", "gato"}
oov = vocab_test - vocab_train # palabras fuera de vocabulario
iou = |A & B| / |A | B| # métrica de visión
```

Fundamental para detectar OOV en NLP, eliminar duplicados en datasets, calcular IoU en detección de objetos y verificar pertenencia rápida.

6. Counter — conteo especializado

Subclase de dict del módulo collections, optimizada para contar elementos hashables.

```
from collections import Counter
distrib = Counter(["gato","perro","gato"])
distrib.most_common(3) # top-3 clases más frecuentes
```

Se usa en NLP para frecuencias de palabras (base de TF-IDF y bag-of-words), detección de desbalanceo de clases y construcción de vocabularios ordenados por frecuencia.

7. collections.deque — cola doblemente terminada

deque (*double-ended queue*) permite **agregar y remover en ambos extremos en $O(1)$** . Internamente es una lista doblemente enlazada por bloques. Reemplaza con ventaja a la list cuando se necesitan operaciones frecuentes en el frente.

```
from collections import deque
buffer = deque(maxlen=1000) # buffer de experiencias en RL
buffer.append(transicion) # tira las viejas automáticamente
batch = random.sample(buffer, 32)
```

En IA la deque brilla en **Reinforcement Learning** como *replay buffer* (DQN), en **ventanas deslizantes** para series temporales, y como cola FIFO en el algoritmo BFS. También es la base recomendada para implementar pilas y colas en Python.

8. Pila (Stack) — política LIFO

Una **pila** es una estructura lineal donde el último en entrar es el primero en salir (**LIFO**). Operaciones fundamentales: push (apilar) y pop (desapilar), ambas $O(1)$.

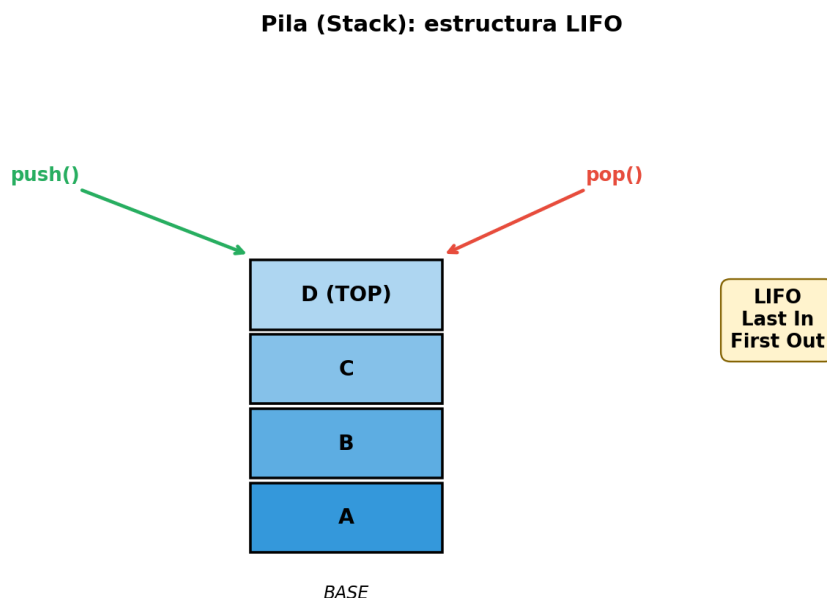


Figura 2. Pila: política LIFO.

```
pila = []
pila.append(x) # push
pila.pop() # pop → devuelve el último insertado
```

En IA la pila aparece en: (i) **DFS iterativo** sobre grafos y árboles, (ii) **backtracking** en agentes de búsqueda y en resolución de CSP, (iii) **undo/redo** de acciones en RL o edición de imágenes, (iv)

evaluación de expresiones y árboles sintácticos en NLP clásico, y (v) el **call stack** de la recursión detrás de todo algoritmo recursivo (búsqueda alpha-beta, minimax).

9. Cola (Queue) — política FIFO

Una **cola** respeta el orden de llegada: el primero en entrar es el primero en salir (**FIFO**). Con `deque`, `append` y `popleft` son $O(1)$.

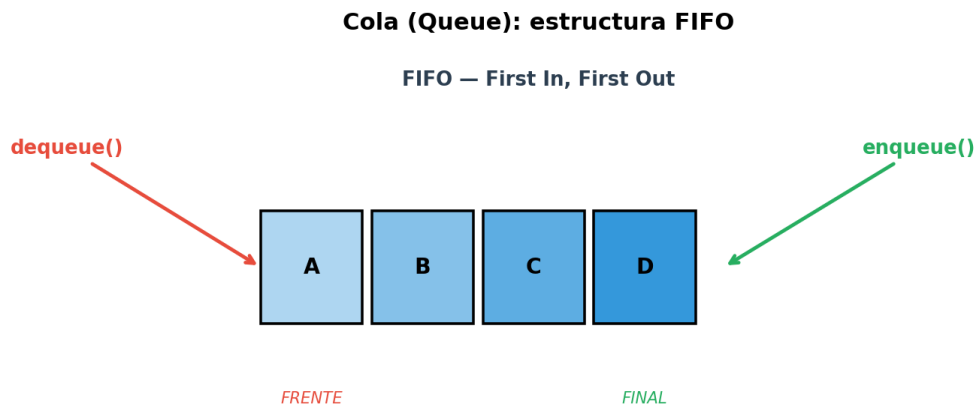


Figura 3. Cola: política FIFO.

```
from collections import deque
cola = deque()
cola.append(x) # enqueue
cola.popleft() # dequeue
```

Usos en IA: (i) motor del algoritmo **BFS**, (ii) colas de trabajos en **pipelines de datos** asíncronos (por ejemplo, `tf.data prefetch`), (iii) planificadores de agentes multitarea, y (iv) buffers de comunicación entre procesos (workers de `DataLoader`).

10. Heap — cola de prioridad

Un **heap** (o montículo) es un árbol binario casi completo con la propiedad de que cada padre es menor (**min-heap**) o mayor (**max-heap**) que sus hijos. Se almacena en un arreglo. Permite obtener el mínimo (o máximo) en $O(1)$ e insertar o extraer en $O(\log n)$.

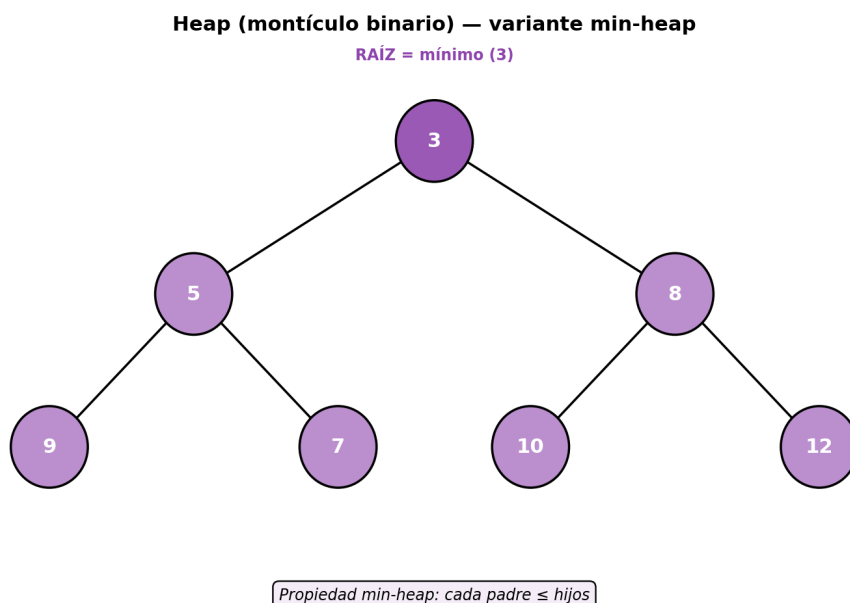


Figura 4. Min-heap: la raíz siempre es el mínimo.

```
import heapq
h = []
heapq.heappush(h, (0.87, "gato"))
heapq.heappush(h, (0.11, "loro"))
prob, clase = heapq.heappop(h) # (0.11, "loro")
```

El heap implementa la **cola de prioridad**, corazón de: (i) algoritmos de búsqueda informada como **A*** y **Dijkstra**, (ii) **beam search** en modelos de lenguaje y traducción automática, (iii) selección eficiente de los **top-k** resultados en sistemas de recomendación y motores de embeddings, y (iv) planificadores de tareas con prioridades en agentes.

11. Árbol binario

Un **árbol binario** es una estructura jerárquica en la que cada nodo tiene como máximo dos hijos. Cuando además cumple la propiedad de ordenación (izquierda < padre < derecha) hablamos de **árbol binario de búsqueda** (BST), con acceso $O(\log n)$ en promedio.

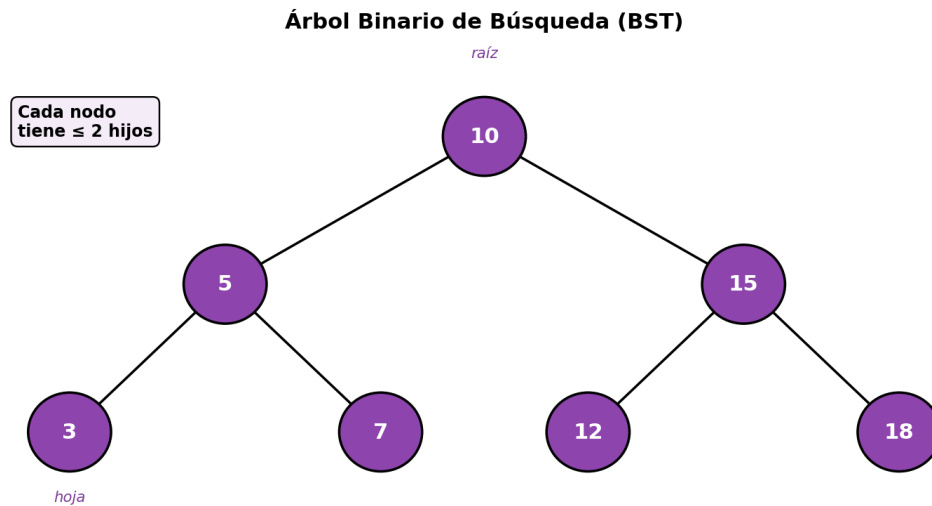


Figura 5. Árbol Binario de Búsqueda.

En IA aparecen como: (i) **árboles de Huffman** para compresión, (ii) **KD-trees** y **Ball-trees** para búsqueda eficiente de vecinos (k-NN) en espacios de baja dimensión, y (iii) estructura recursiva de los **árboles sintácticos** en NLP clásico.

12. Árbol n-ario

En un **árbol n-ario** cada nodo puede tener cualquier número de hijos. Es la generalización natural del árbol binario.

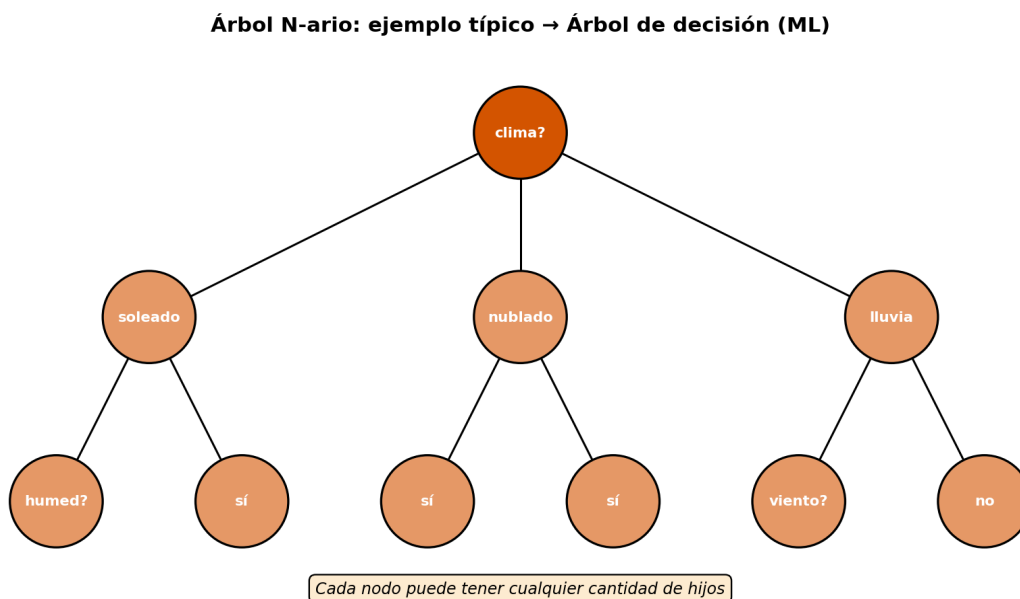


Figura 6. Árbol de decisión: caso típico de árbol n-ario.

Los árboles n-arios son centrales en IA clásica y moderna:

- **Árboles de decisión** (ID3, C4.5, CART) y sus ensambles (**Random Forest**, **XGBoost**, **LightGBM**).
- **MCTS** (Monte Carlo Tree Search): base de AlphaGo, AlphaZero y de agentes en juegos.
- **Árboles de juego** (minimax, alpha-beta) en ajedrez y videojuegos.
- **Trie** para tokenización, autocompletado y estructuras de búsqueda de subcadenas.

13. Grafos

Un **grafo** $G = (V, E)$ es un conjunto de **vértices** V conectados por **aristas** E . Puede ser dirigido o no dirigido, con pesos o sin ellos. Se representa habitualmente con lista de adyacencia (dict de sets), matriz de adyacencia (NumPy), o bibliotecas dedicadas como networkx y PyTorch Geometric.

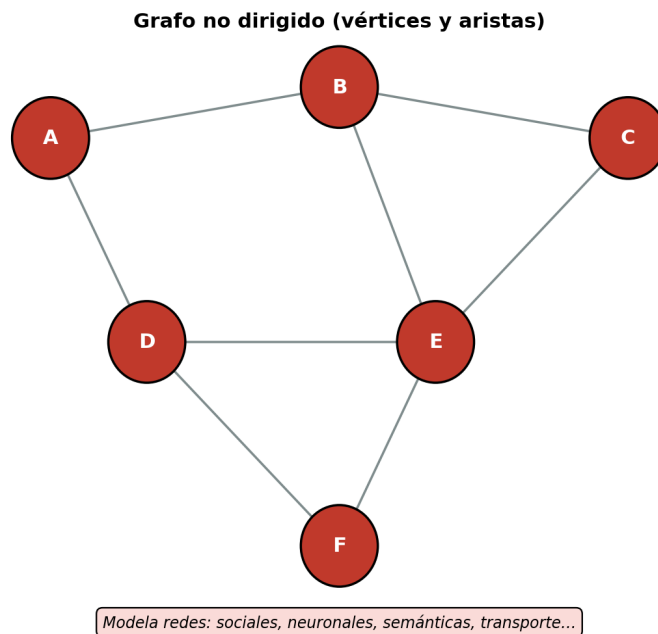


Figura 7. Grafo no dirigido representando una red.

```
grafo = {  
    "A": {"B", "D"},  
    "B": {"A", "C", "E"},  
    # ...  
}
```

Los grafos modelan casi todo en IA moderna: (i) **redes sociales** y sistemas de recomendación, (ii) **Knowledge Graphs** y bases de conocimiento, (iii) **redes neuronales** mismas (grafos de cómputo de PyTorch/TensorFlow), (iv) **Graph Neural Networks** (GCN, GAT, GraphSAGE) para química, biología y NLP, (v) rutas y planificación de agentes, y (vi) modelos probabilísticos gráficos (redes bayesianas).

14. Recorridos de grafos: BFS y DFS

Los dos algoritmos fundamentales para explorar árboles y grafos son **BFS** (búsqueda en anchura) y **DFS** (búsqueda en profundidad). Comparten la misma estructura general, pero difieren en el orden de visita y en la estructura auxiliar que emplean.

BFS usa cola (FIFO) — DFS usa pila (LIFO) o recursión

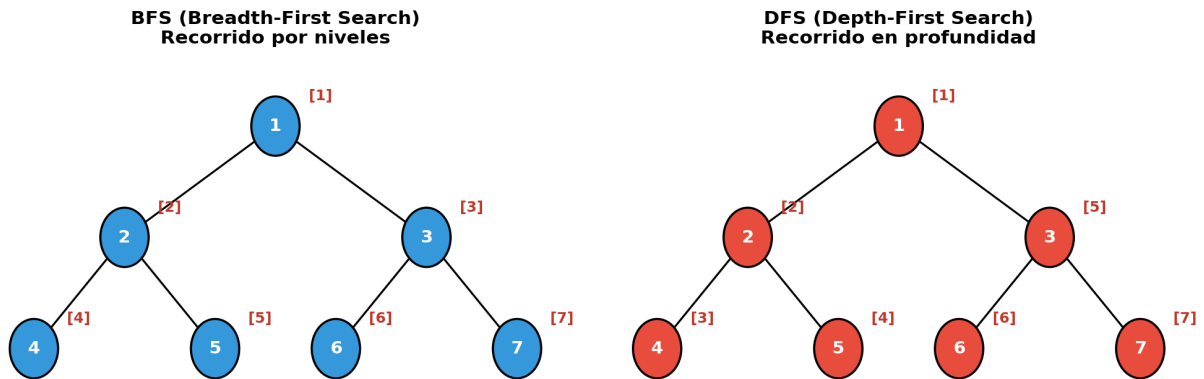


Figura 8. Orden de visita en BFS vs DFS.

BFS — Breadth-First Search

Explora todos los vecinos del nivel actual antes de bajar al siguiente. Usa una **cola (FIFO)**. Encuentra el **camino más corto** en grafos no ponderados.

```
from collections import deque
def bfs(grafo, inicio):
    visitado = {inicio}
    cola = deque([inicio])
    while cola:
        nodo = cola.popleft()
        for vecino in grafo[nodo]:
            if vecino not in visitado:
                visitado.add(vecino)
                cola.append(vecino)
```

DFS — Depth-First Search

Baja lo más profundo posible antes de retroceder. Usa una **pila (LIFO)** —explícita o vía recursión—. Es la base del backtracking, de la detección de ciclos y del ordenamiento topológico.

```
def dfs(grafo, nodo, visitado=None):
    if visitado is None: visitado = set()
    visitado.add(nodo)
    for vecino in grafo[nodo]:
        if vecino not in visitado:
            dfs(grafo, vecino, visitado)
```

BFS vs DFS en IA

Criterio	BFS	DFS
Estructura auxiliar	Cola (deque)	Pila / recursión
Camino más corto (no ponderado)	Si	No

Uso de memoria	Alto (nivel completo)	Bajo (una rama)
Usos típicos en IA	Búsqueda óptima, GNN por capas	Backtracking, MCTS, DFID

15. Dataset

Un **Dataset** es una colección de ejemplos usados para entrenar, validar o evaluar un modelo. Los frameworks lo abstraen en clases:

Framework	Clase	Propósito
PyTorch	<code>torch.utils.data.Dataset</code>	Interfaz <code>__len__</code> y <code>__getitem__</code>
TensorFlow	<code>tf.data.Dataset</code>	Pipeline lazy y paralelo
Hugging Face	<code>datasets.Dataset</code>	Backend Arrow, eficiente
scikit-learn	<code>sklearn.datasets</code>	Datasets clásicos preempaquetados

```
class MiDataset(Dataset):
    def __len__(self): return len(self.X)
    def __getitem__(self, i): return self.X[i], self.y[i]
```

16. DataFrame

Estructura tabular 2D de pandas (o Polars, Dask, PySpark). Cada columna es un arreglo NumPy o Arrow contiguo, lo que permite operaciones vectorizadas muy rápidas.

```
import pandas as pd
df = pd.read_csv("clientes.csv")
df["edad_norm"] = (df.edad - df.edad.mean()) / df.edad.std()
resumen = df.groupby("segmento")["ingreso"].mean()
```

Es la herramienta reina en **EDA y preprocesamiento**: limpieza, imputación de faltantes, codificación de categóricas, joins, agregaciones y feature engineering. Antes del entrenamiento se convierte típicamente a NumPy o a un tensor.

17. Tabla comparativa integral

Estructura	Tipo	Acceso	Búsqueda	Uso en IA
list	lineal	$O(1)$	$O(n)$	Secuencias, batches
tuple	lineal	$O(1)$	$O(n)$	Pares (x,y), shapes
dict	hash	$O(1)$	$O(1)$	Configs, vocab, state_dict
set	hash	—	$O(1)$	OOV, IoU, unicidad
Counter	hash	$O(1)$	$O(1)$	Frecuencias, desbalanceo
deque	lineal	$O(1)^\dagger$	$O(n)$	Replay buffer, BFS
pila	lineal	$O(1)^\ddagger$	$O(n)$	DFS, backtracking, undo
cola	lineal	$O(1)^\ddagger$	$O(n)$	BFS, pipelines de datos
heap	árbol	$O(1)^*$	$O(n)$	A*, beam search, top-k
árbol bin.	árbol	$O(\log n)$	$O(\log n)$	KD-tree, Huffman, k-NN
árbol n-ario	árbol	$O(\log n)$	$O(\log n)$	Decision trees, MCTS, tries
grafo	no lineal	$O(1)^\S$	$O(V+E)$	GNN, KG, redes sociales
Dataset	abstracción	$O(1)$	—	Fuente de ejemplos
DataFrame	tabular	$O(1)^\P$	$O(n)$	EDA, preprocesamiento

† en ambos extremos. ‡ sólo en la cima/frente. * al mínimo/máximo (raíz). § acceso a lista de adyacencia. ¶ acceso columnar.

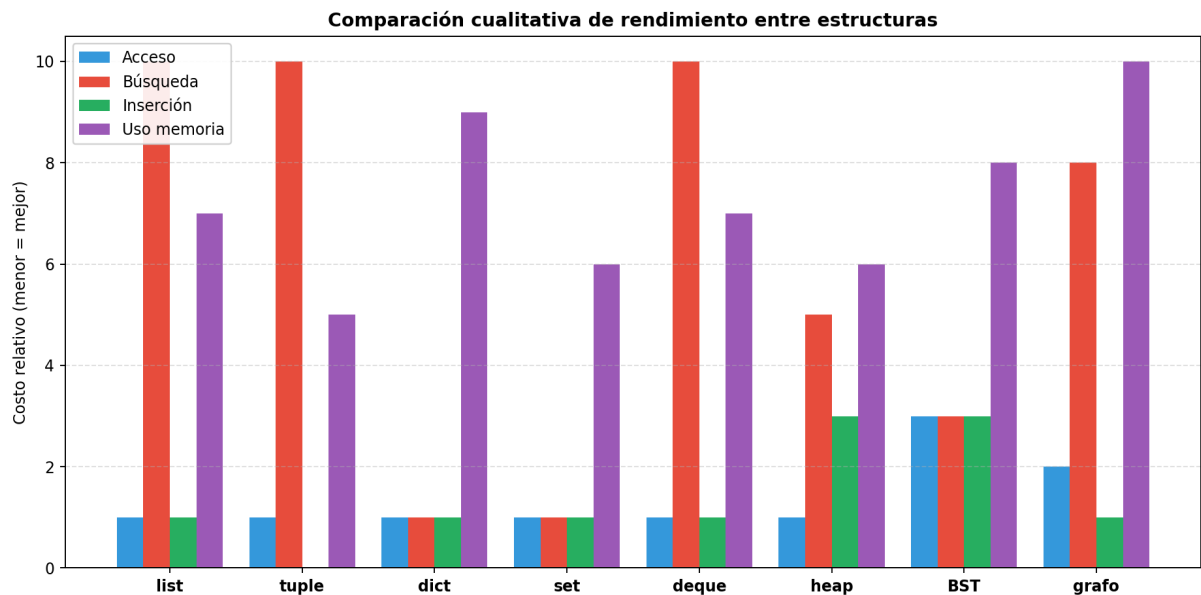


Figura 9. Comparación cualitativa de rendimiento.

18. ¿Cuándo se usa cada estructura?

En un proyecto real de IA las estructuras no compiten: se combinan. Cada etapa privilegia unas u otras según sus necesidades. La figura muestra un panorama completo que incluye tanto la fase de datos como la fase de razonamiento (típica en agentes y sistemas de búsqueda):

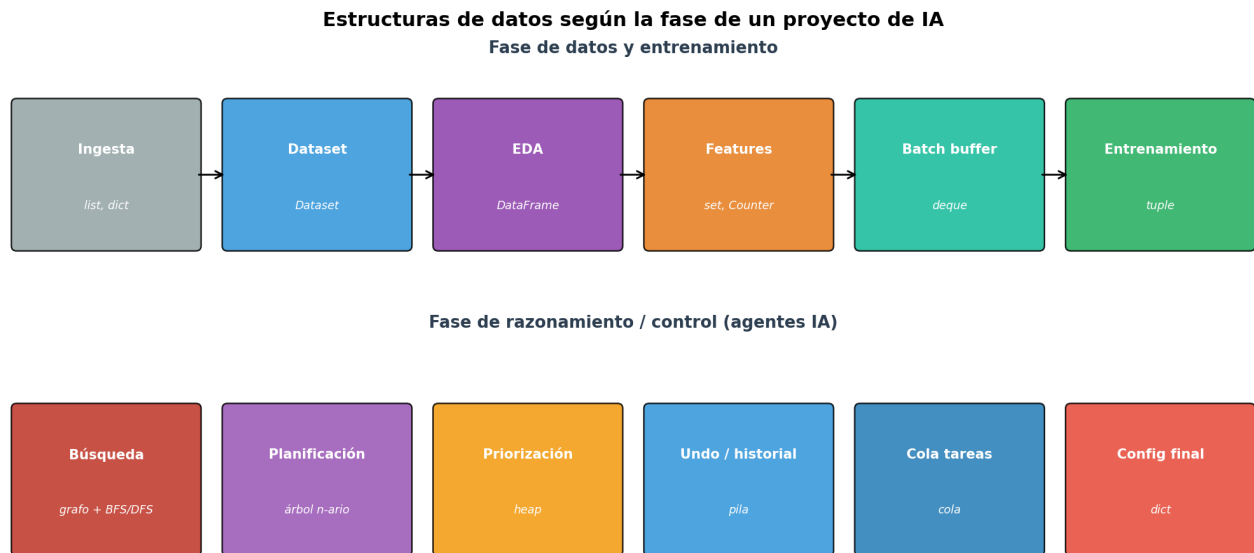


Figura 10. Estructuras según la fase del proyecto de IA.

19. ¿Cuál es la “mejor” estructura?

No existe una estructura universalmente mejor: existe la más adecuada para cada problema. La regla de oro es **elegir la estructura cuyo modelo de acceso natural coincide con la operación dominante del algoritmo**.

Situación	Estructura recomendada
Necesitas orden y modificación in-place	list
Datos inmutables y quizás clave de un dict	tuple
Mapeo clave-valor rápido	dict
Sólo importa pertenencia y unicidad	set
Contar frecuencias	Counter
Ventana deslizante o replay buffer	deque
Recursión, backtracking, undo/redo	pila
BFS o pipeline productor-consumidor	cola
Extraer siempre el más prioritario (A*, top-k)	heap
Búsqueda ordenada en bajas dimensiones	árbol binario / KD-tree
Modelo de decisión o búsqueda en juegos	árbol n-ario
Relaciones arbitrarias entre entidades	grafo

Entrenar un modelo con muchos ejemplos	Dataset
EDA y preprocesamiento tabular	DataFrame

20. Conclusiones

Este recorrido muestra que el trabajo en IA descansa sobre un conjunto de estructuras heredadas de la ciencia de la computación clásica. Las tres grandes lecciones para el estudiante son:

1. Las estructuras **lineales** (list, tuple, deque, pila, cola) resuelven el 80% del trabajo cotidiano en scripts de preprocesamiento y control de flujo.
2. Las estructuras **indexadas por hash** (dict, set, Counter) son la clave para escalar en tiempo: sin ellas, muchos algoritmos de NLP y ML serían inviables.
3. Las estructuras **jerárquicas y no lineales** (heap, árboles, grafos) habilitan el razonamiento simbólico, la búsqueda informada y las modernas Graph Neural Networks.

Ejercicios integradores propuestos para el estudiante:

- Implementar BFS y DFS sobre el mismo grafo y comparar los caminos encontrados.
- Construir un *replay buffer* con **deque(maxlen=N)** para un agente de RL.
- Implementar A* usando **heapq** como cola de prioridad.
- Comparar empíricamente el tiempo de búsqueda en list vs set con 10⁶ elementos.
- Entrenar un DecisionTreeClassifier y visualizar el árbol n-ario resultante.
- Modelar una red social como grafo con networkx y aplicarle una GCN con PyTorch Geometric.

Fin del informe. Cualquier duda, discusión en la próxima clase.